Does Model Compression Undo Robustness?

Group robustness of compressed vision transformers,
and whether data pruning survives deployment

Anusha Upadhyay Shreyas Visweshwaran Swasti Sadanand

Advisor: Prof. Jung-Eun Kim

September 21, 2026

This plan covers the scope approved on 7 September 2025: *both* the first phase (compression vs. group robustness) and the second phase (does the robustness bought by data pruning survive compression). Phase 1 is complete and its results are reported in §3. Phases 2 and 3 are specified in §4–§5.

1 Approved scope

The question is whether the two things we routinely do to a model in sequence — first make it robust, then make it small — interact badly. A robustified model is compressed before deployment, and we want to know whether that second step silently undoes the first: whether worst-group accuracy degrades faster under compression than average accuracy does, so that a model still looking healthy on top-1 has quietly returned to relying on the spurious feature.

Two phases were approved:

Phase 1
Fine-tune ViT-S/16 on Waterbirds, apply post-hoc compression (attention-head pruning, FFN neuron pruning, unstructured magnitude pruning, and weight quantization) with no retraining afterwards, and measure worst-group against average accuracy.

Phase 2
Apply the data pruning of Mulchandani and Kim (ICLR 2025) *before* compression, and test whether the robustness it buys survives.

We add a third phase, §5, which the original proposal committed to conditionally: if Phase 1 found a robustness gap, we would test a fix by recomputing core-feature difficulty on the compressed model rather than the dense one. Phase 1 found the gap, so the condition is met.

2 Setup

Model	`vit_small_patch16_224` (timm), ImageNet-pretrained, 2-way head
Data	Waterbirds v1.0, official splits, read directly from `metadata.csv`
Training	Plain ERM, 10 epochs, AdamW lr $= 10^{-5}$, cosine, batch 32
Selection	Best *validation average* accuracy (seed 0: epoch 3)
Compression	Applied to the trained checkpoint, evaluated immediately, **never retrained**
Scope	In-block Linear weights only: `attn.qkv`, `attn.proj`, `mlp.fc1`, `mlp.fc2` (21,233,664 params)

Four groups, indexed $g = 2y +$ background: `landbird_on_land` (majority, $n_{\text{train}} = 3498$), `landbird_on_water` (minority, 184), `waterbird_on_land` (minority, 56), `waterbird_on_water` (majority, 1057). The two minority groups are the ones that *conflict* with the background shortcut.

Three choices that decide what this measures

Training is deliberately group-blind. No reweighting, no balanced sampler, no GroupDRO. The model is *supposed* to learn the background shortcut — if training were group-balanced, worst-group accuracy would start high and the experiment would measure nothing.

Model selection is on validation average accuracy, never worst-group. Selecting on worst-group is itself a group-aware intervention; it would inflate the ERM baseline and void the premise. Adjusted-average selection is out for the same reason — the adjustment weights need group labels, so using it would leak group information into a pipeline meant to have none.

No retraining after compression. Fine-tuning afterwards lets the model re-fit the shortcut, and answers a different question.

Throughout, `acc_adj_avg` is per-group accuracy reweighted by *training* frequency. Waterbirds' val/test splits are far more group-balanced than train, so a plain test-set mean is not the ∼97% figure quoted in the literature; the adjusted average is.

3 Phase 1: results (complete)

Baseline, uncompressed, seed 0: **98.2% adjusted average, 70.4% worst group**, a gap of 27.8 points. Twenty-nine compression configurations were evaluated against that checkpoint; thirty-one validation gates guard the pipeline.

Configuration	Sparsity	Adjusted avg	Worst group	Gap
uncompressed	0.000	98.2%	70.4%	27.8
quantize 8-bit, per-channel	0.014	98.2% (−0.0)	70.4% (−0.0)	27.8
quantize 6-bit, per-tensor	0.190	97.8% (−0.4)	62.6% (−7.8)	35.2
magnitude, 30%	0.300	97.5% (−0.8)	54.4% (−16.0)	43.1
quantize 4-bit, per-channel	0.232	97.1% (−1.2)	60.6% (−9.8)	36.5
remove 6 of 72 heads	0.028	96.5% (−1.7)	48.1% (−22.3)	48.4
FFN, 30% of neurons	0.200	83.3% (−14.9)	2.5% (−67.9)	80.9

Table 1: Compression at matched sparsity costs far more worst-group accuracy than average accuracy. Deltas are against the uncompressed baseline, in points.

The headline. Pruning 30% of the transformer's weights costs **0.8 points** of adjusted average accuracy and **16.0 points** of worst-group accuracy — a 21× difference. An evaluation reporting only average accuracy would call that compression free. Removing **6 attention heads out of 72**, 2.8% of the targeted weights, costs 1.7 points of average accuracy and **22.3 points** of worst-group accuracy.

Two things in this data that are easy to misread

The gap narrows again at extreme compression, and that is not robustness returning. Past roughly 80% magnitude sparsity or 70% FFN removal the model collapses to predicting a single class. Accuracy is still well-defined, but a model with one output has no group disparity left to measure. Those points are detected by the `pred_mean` collapse gate, drawn hollow in every figure, and excluded from the trade-off plot.

Head removal is not monotonic. Removing 48 heads scores *better* on worst-group (34.1%) than removing 36 (1.1%), then falls again at 60. Global magnitude ranking does not remove heads

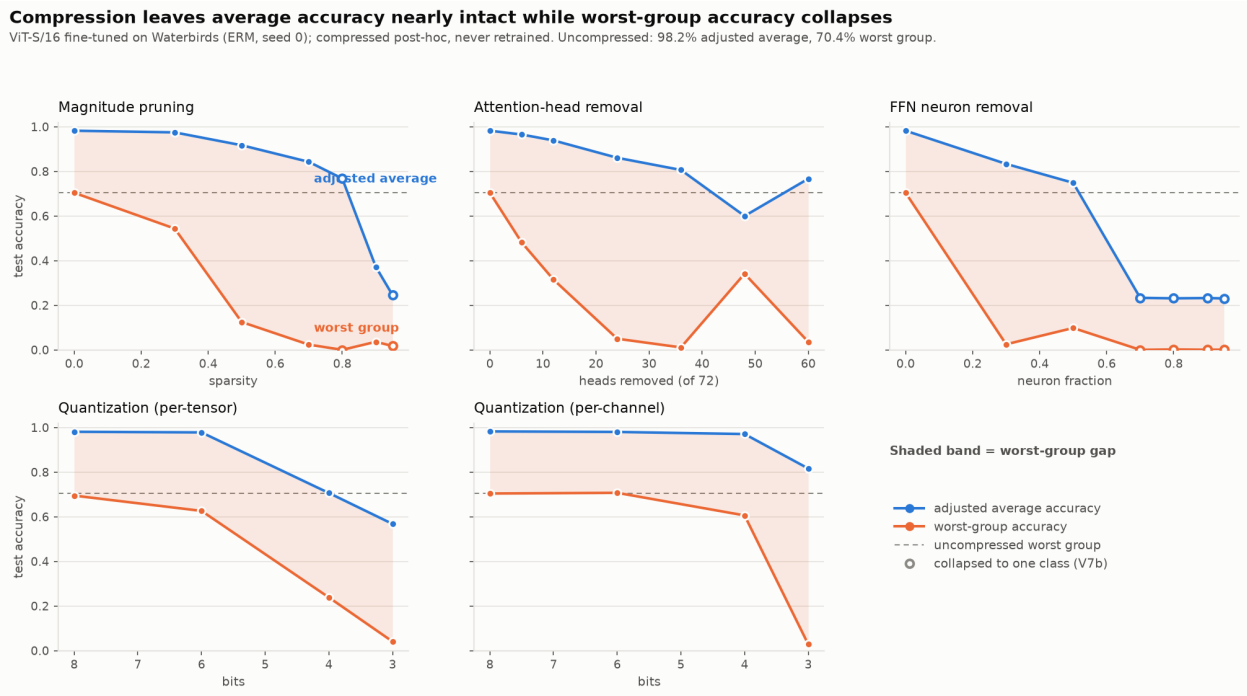


Figure 1: Average vs. worst-group accuracy across all five compression families. The shaded band is the gap; it widens everywhere. Hollow markers are configurations where the model has collapsed to a single predicted class.

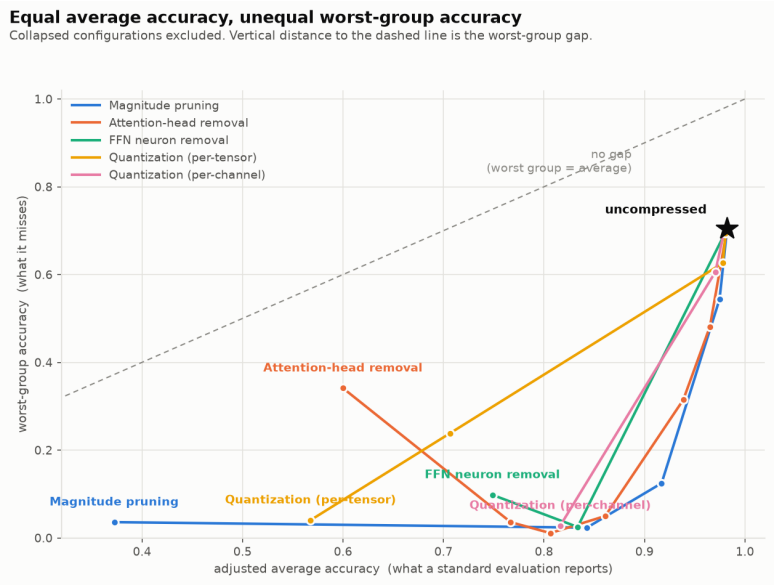


Figure 2: Worst-group plotted against average accuracy, so methods are comparable at equal average accuracy despite non-commensurable knobs. Collapsed configurations are excluded.

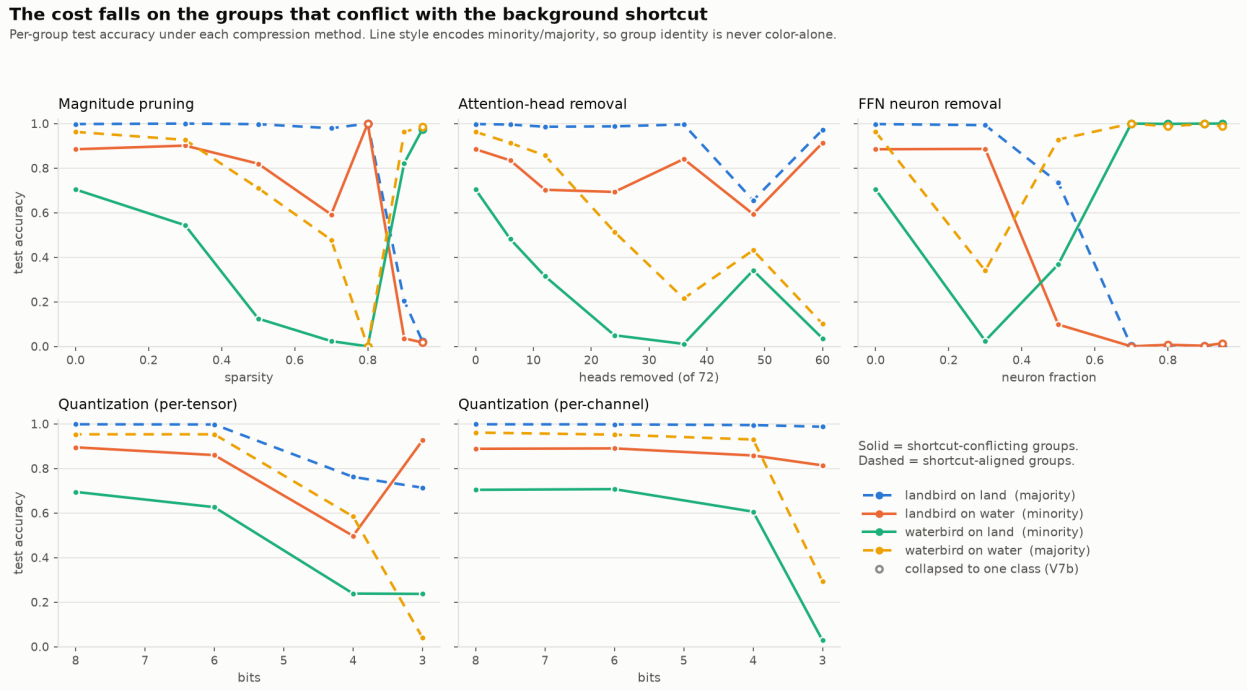


Figure 3: Which groups pay. `landbird_on_land` — the majority group aligned with the shortcut — holds near 100% throughout, while `waterbird_on_land`, the 56-sample conflicting minority, collapses first.

in order of behavioral importance, and near-degenerate models reshuffle which group is worst. The deep end of the head sweep is noise around a collapsed model, not signal.

Quantization granularity matters more than bit width. Per-channel 4-bit keeps 60.6% worst-group; per-tensor 4-bit keeps 23.8%. Same bit budget, very different outcome.

4 Phase 2: does data pruning survive compression?

Phase 1 establishes that compression damages the worst group disproportionately on a model that was never made robust in the first place. Phase 2 asks the question that matters for deployment: if we *do* make the model robust first, does compression take it back?

4.1 The intervention

We use the data pruning of Mulchandani and Kim, *Severing Spurious Correlations with Data Pruning* (ICLR 2025). Its premise is that spurious correlations are learned primarily because of a handful of training samples whose *core* feature is hard to learn: where the core feature is hard, the network prefers the simpler spurious one. Removing those samples severs the dependency, with no group labels, no inferred domain knowledge, and no human intervention — which is exactly what our group-blind pipeline requires.

Sample difficulty is the L2 distance between the network's predicted distribution and the one-hot label, read from a single ERM run at a fixed early epoch:

$$d_i \;=\; \| \, p(w_e, x_i) - y_i \, \|_2,$$

with $e = 1$ for Waterbirds in the original work. Samples are ranked by d_i, the *hardest* are removed class-balanced, and the network is retrained from scratch on what remains.

Two deviations from the paper, both deliberate and both to be stated in any write-up. The paper's Waterbirds experiments use an ImageNet-pretrained ResNet-50; we use ViT-S/16, because the architectural question — whether robustness degrades faster when attention heads are removed than when FFN neurons are — is the contribution and requires a transformer. And we hold the training recipe fixed at our Phase 1 recipe rather than the paper's, so that the ERM and pruned models differ in exactly one thing: the training set.

4.2 What we add to the pipeline

Phase 1's code is reusable almost entirely; the compression methods, metrics, caching, and sweep harness do not change. Five additions are needed:

1. A **train-split uint8 cache**. `configs/base.yaml` already declares `paths.train_cache`, but `src/data.py` currently builds only the val and test caches — training reads JPEGs through random augmentation every epoch. Difficulty scoring needs a deterministic, un-augmented pass over the training split, which is what this cache provides.
2. **Epoch-1 score capture** in `src/train.py`, which today saves a checkpoint only when validation average accuracy improves and never records per-sample outputs.
3. A new `src/prune_data.py`: computes d_i for all 4,795 training samples, ranks them, and emits class-balanced keep-index sets — the analogue of the reference implementation's `prune_general.py`, the variant that assumes no access to spurious information.
4. A `--prune-indices` path through `train_one_seed`, so a retrain on a pruned subset is the same code path with a subset sampler.
5. A `--checkpoint` argument on `src/sweep.py`, which currently hard-codes `vit_s16_waterbirds_seed{seed}.pt`. Phase 2 needs the same 29-configuration sweep pointed at a second checkpoint.

Every existing validation gate continues to apply, and two are added: that the pruned training set is class-balanced to within one sample, and that the kept indices are a strict subset of the original training split.

4.3 Experimental matrix

Pruning fractions k	$\{1, 2, 5, 10\}$% of the training set, class-balanced, hardest-first
Seeds	$\{0, 1, 2\}$, as `configs/base.yaml` already declares
Selection	Unchanged: best validation *average* accuracy, group-blind
Compression	The same 29 configurations as Phase 1, on the robustified checkpoint
Comparison	ERM checkpoint vs. pruned checkpoint, at matched achieved sparsity

Phase 2 proceeds in two gated steps. First we confirm the intervention works at all on our setup — that the pruned model's dense worst-group accuracy exceeds the ERM model's, averaged over three seeds. Only if it does is the compression sweep meaningful; if data pruning buys nothing here, there is nothing for compression to undo, and that null result is reported as such rather than swept under a sparsity curve.

4.4 The measurement

The headline quantity is the **retention ratio**. Let $\Delta(s)$ be the worst-group accuracy of the pruned model minus that of the ERM model, both compressed to achieved sparsity s. Then

$$R(s) \;=\; \frac{\Delta(s)}{\Delta(0)}$$

is the fraction of the dense robustness gain still present after compression. $R(s) \approx 1$ means the intervention survives deployment. $R(s) \approx 0$ means compression silently undid it — a model that was made robust, then made small, and is no longer robust, while its average accuracy barely moved.

We report $R(s)$ per compression family, because Phase 1 already shows the families are not interchangeable: 6 attention heads out of 72 cost more worst-group accuracy than 30% unstructured sparsity does. If the retention ratio differs across families, then *which* compression method a practitioner chooses determines whether their robustness intervention was worth running, which is a directly actionable finding.

Secondary measurements: per-group accuracy curves (which group pays, and whether pruning changes the answer), the adjusted-average cost of pruning at $s = 0$ (the price of the intervention), and the collapse gate on every row.

4.5 Both outcomes are reportable

We are not assuming a direction. Hooker et al. found that compression disproportionately harms atypical examples in CNNs; Phase 1 confirms the pattern for ViTs on a group-robustness benchmark. If the data-pruning gain also fails to survive, we have shown that a state-of-the-art robustness intervention has a deployment-time expiry date, and §5 offers a fix. If the gain *does* survive, we have shown that intervening on the data buys something more durable than intervening on the objective — which is a point in the method's favour and equally worth reporting.

5 Phase 3: compression-aware data pruning

If Phase 2 finds that the robustness gain evaporates, the diagnosis is straightforward: core-feature difficulty was measured on the *dense* model, but the model that gets deployed is the *compressed* one, and the two do not find the same samples hard. A sparse network has less capacity for exactly the hard core features whose difficulty drives the spurious shortcut, so the sample set that should have been pruned is not the one we pruned.

The fix is to move the measurement. We compress the epoch-1 ERM checkpoint to the deployment target, compute d_i on *that* model, prune class-balanced hardest-first as before, retrain the dense model on the resulting subset, and then compress it. The pruning decision is thereby made with respect to the model that will actually be shipped.

Two design points, stated explicitly so they are not quietly chosen later:

- **Which configuration defines the deployment target.** We propose magnitude pruning at 30% sparsity — the mildest setting in Phase 1 that already shows a 16-point worst-group drop while costing under one point of average accuracy, i.e. precisely the regime a practitioner would consider safe. If Phase 2's retention ratio turns out to be much worse for head pruning, we additionally target 6-head removal.

- **Cost.** Scoring on a compressed model needs no new training run beyond the retrain: the epoch-1 checkpoint already exists from the Phase 2 ERM run, compression is a forward-pass-only operation, and the scoring pass over 4,795 cached training images takes well under a minute. Phase 3 is one extra retrain plus one extra sweep per seed.

The comparison is direct: $R(s)$ under compression-aware pruning against $R(s)$ under the dense-scored pruning of Phase 2, same seeds, same sweep, same metric.

6 Threats to validity

Seed variance is the main one. Phase 1 rests on a single seed. Phase 2 compares two *training runs*, so a worst-group difference between two single runs cannot be distinguished from seed noise — worst-group accuracy on Waterbirds is measured on a 642-sample group and is volatile. This is why the plan commits to three seeds, and why we will report per-seed numbers alongside means rather than means alone. Phase 1's headline gaps (16–22 points) are far larger than typical seed variance, but the Phase 2 effect may not be.

The 56-sample group. `waterbird_on_land` has 56 training samples and is the worst group in nearly every Phase 1 row. Class-balanced pruning at $k = 10\%$ removes roughly 480 samples overall; if a disproportionate share comes from that group — plausible, since minority-group samples are exactly the ones with hard core features — the intervention could damage the very group it targets. We will report the per-group composition of every pruned set, and treat a large draw from group 2 as a finding rather than a nuisance.

Limited headroom. Our ERM worst-group accuracy is 70.4%, higher than the ResNet-50 Waterbirds figures typically quoted. There may simply be less for data pruning to recover here than in the original paper's setting, which would compress the Phase 2 effect size. We state this before seeing the result, not after.

Simulated quantization. Quantization is quantize–dequantize. The measurement here is accuracy, not latency: the numerical effect is reproduced exactly, and real int4/int8 kernels are CUDA-only and irrelevant to an accuracy study.

Scope. One dataset, one architecture. Waterbirds is a synthetic-shortcut benchmark with a small, clean spurious correlation; nothing here establishes the effect size on natural distribution shift. Heads and neurons are ranked by weight L2 norm, a data-free criterion; a calibration-data criterion might prune in a more robustness-preserving order, and testing that is the obvious follow-on experiment if time permits.

7 Compute budget and schedule

Everything runs on Apple-silicon MPS on our own machines. A clean 10-epoch fine-tune is roughly 20 minutes; one 29-configuration sweep is roughly 16 minutes.

The work is stageable overnight: `src/sweep.py` appends and flushes each row and skips configurations already present in `results/sweep.csv`, so an interrupted run resumes where it stopped rather than restarting.

8 Division of work

Three workstreams, one owner each, with the whole team on analysis and writing. The split below is our proposal and we will confirm it among ourselves before starting.

Work item	Runs	Wall clock
Train-split cache + difficulty scoring	3	∼1 h
Phase 2: ERM baselines, seeds 1–2 (seed 0 exists)	2	∼0.7 h
Phase 2: pruned retrains, 4 fractions × 3 seeds	12	∼4 h
Phase 3: compression-aware retrains, 3 seeds	3	∼1 h
Compression sweeps on selected checkpoints	6–9	∼2 h
Figures, tables, validation gates	—	∼1 h
Total (with slack for machine contention)		**12–15 h**

Workstream	Owner	Content
Data pruning	Anusha Upadhyay	Train cache, difficulty scoring, `prune_data.py`, retrains
Compression & metrics	Shreyas Visweshwaran	Sweep extension, retention ratio, validation gates
Analysis & write-up	Swasti Sadanand	Figures, per-group composition, threats, final report

References

1. V. Mulchandani and J.-E. Kim. *Severing Spurious Correlations with Data Pruning.* ICLR 2025 (Spotlight). arXiv:2503.18258.
2. S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang. *Distributionally Robust Neural Networks for Group Shifts.* ICLR 2020. (Waterbirds, worst-group accuracy, adjusted average.)
3. S. Hooker, A. Courville, G. Clark, Y. Dauphin, and A. Frome. *What Do Compressed Deep Neural Networks Forget?* arXiv:1911.05248, 2019.
4. P. W. Koh et al. *WILDS: A Benchmark of in-the-Wild Distribution Shifts.* ICML 2021.